ITEMFLIX

Full-Stack REST API Project

Complete Code Explanation for Staff

---

1. What is ITEMFLIX?

ITEMFLIX is a full-stack web application that allows users to manage a list of items (like products). It is built using two parts:

- Backend: Python Flask REST API — handles data storage and logic
- Frontend: React.js — the visual interface users interact with
- Database: SQLite — stores all item data permanently

Think of it like this:

- Netflix shows movies → ITEMFLIX shows Items
- You can Add, View, Edit and Delete items
- All data is saved in a database

2. Project Structure

Here is how the project folders are organized:

File / Folder	What it does
backend/app.py	Main Flask server file — all API routes (GET, POST, PUT, DELETE)
backend/models.py	Defines the Item data structure (name, price, category etc.)
backend/database.py	Sets up the SQLite database connection
backend/requirements.txt	List of Python packages needed to run the backend

backend/Procfile	Tells Render.com how to start the server
frontend/src/App.jsx	Main React file — all UI components and logic in one file
frontend/src/index.css	CSS styles including Tailwind CSS imports
frontend/src/main.jsx	Entry point — starts the React app
frontend/tailwind.config.js	Tailwind CSS configuration for dark mode and colors
frontend/postcss.config.js	PostCSS configuration for Tailwind to work

3. What is a REST API?

REST API stands for Representational State Transfer Application Programming Interface.

In simple words:

- It is a way for the Frontend (React) to talk to the Backend (Flask)
- They communicate using HTTP requests over the internet
- Data is sent and received in JSON format

REST API uses 4 main operations called CRUD:

Operation	HTTP Method	What it does	Example URL
CREATE	POST	Add a new item	/items
READ	GET	Get all items or one item	/items or /items/1
UPDATE	PUT	Edit an existing item	/items/1
DELETE	DELETE	Remove an item	/items/1

4. How REST API Works in ITEMFLIX

Here is the complete flow of how data moves in this project:

Step 1 — User Opens the App

- User opens https://item-manager-rest-api.vercel.app in browser
- React app loads on screen (ITEMFLIX UI)
- React automatically sends a GET request to the Flask API

Step 2 — GET Request (Read All Items)

```
React sends:  GET https://itemflix-api.onrender.com/items
Flask returns: { success: true, data: [ {id:1, name:'mohan'...} ] }
```

- React receives the JSON data
- React displays items as Netflix-style cards on screen

Step 3 — POST Request (Add New Item)

- User clicks '+ Add' button → form opens
- User fills: Name, Description, Category, Price
- User clicks 'Add Title' → React sends POST request

```
React sends:  POST https://itemflix-api.onrender.com/items
             Body: { name: 'iPhone', category: 'Electronics', price:
75000 }
Flask returns: { success: true, message: 'Item created successfully' }
```

- React refreshes the list → new item card appears

Step 4 — PUT Request (Update Item)

- User hovers card → clicks Edit (pencil) button
- Form opens with existing data pre-filled
- User changes values → clicks 'Update'

```
React sends:  PUT https://itemflix-api.onrender.com/items/1
             Body: { name: 'Updated Name', price: 9999 }
Flask returns: { success: true, message: 'Item updated successfully' }
```

Step 5 — DELETE Request (Remove Item)

- User hovers card → clicks Delete (trash) button
- Confirmation popup appears → user confirms

```
React sends:  DELETE https://itemflix-api.onrender.com/items/1
Flask returns: { success: true, message: 'Item deleted successfully' }
```

- Card disappears from the screen

5. Backend Code Explanation

5.1 app.py — The Main Flask File

This is the most important backend file. It contains all the API routes (endpoints).

Imports and Setup

```
import os
from flask import Flask, request, jsonify
from flask_cors import CORS
from database import db, init_database
from models import Item
```

Import	Purpose
Flask	The main web framework that creates the API server
request	Gets data sent from the frontend (JSON body, URL params)
jsonify	Converts Python dictionary to JSON response
CORS	Allows frontend (React) to call the backend API
Item	The database model for our items

CORS Configuration

```
CORS(app)
```

CORS (Cross-Origin Resource Sharing) allows the React frontend running on Vercel to make API calls to the Flask backend on Render. Without this, the browser would block all requests for security reasons.

GET All Items Route

```
@app.route('/items', methods=['GET'])
def get_items():
    items = Item.query.all()
    return success_response([item.to_dict() for item in items])
```

This route runs when React sends GET /items. It fetches all items from the database and returns them as a JSON list.

POST Create Item Route

```
@app.route('/items', methods=['POST'])
def create_item():
    data = request.get_json()
    new_item = Item(name=data['name'], price=data['price']...)
    db.session.add(new_item)
    db.session.commit()
```

This route runs when React sends POST /items with item data. It validates the data, creates a new Item in the database, and returns the created item.

PUT Update Item Route

```
@app.route('/items/<int:item_id>', methods=['PUT'])
def update_item(item_id):
    item = Item.query.get(item_id)
    item.name = data.get('name', item.name)
    db.session.commit()
```

The <int:item_id> in the URL captures the item ID from the URL. For example, PUT /items/3 sets item_id to 3. It finds that item and updates its fields.

DELETE Item Route

```
@app.route('/items/<int:item_id>', methods=['DELETE'])
def delete_item(item_id):
    item = Item.query.get(item_id)
    db.session.delete(item)
    db.session.commit()
```

Finds the item by ID, removes it from the database, and returns a success response.

5.2 models.py — The Item Data Model

This file defines what an 'Item' looks like in the database — like a template or blueprint.

```
class Item(db.Model):
    id          = db.Column(db.Integer, primary_key=True)
    name        = db.Column(db.String(100), nullable=False)
    description = db.Column(db.String(500))
    category    = db.Column(db.String(50))
    price       = db.Column(db.Float)
    created_at  = db.Column(db.DateTime)
    updated_at  = db.Column(db.DateTime)
```

Field	Type	Description
id	Integer	Unique number for each item (auto-generated)
name	String	Name of the item (required, max 100 chars)
description	String	Optional description (max 500 chars)
category	String	Category like Electronics, Books etc.
price	Float	Price in Rupees (decimal number)
created_at	DateTime	Date and time when item was added
updated_at	DateTime	Date and time when item was last edited

5.3 database.py — Database Setup

```
from flask_sqlalchemy import SQLAlchemy
db = SQLAlchemy()
```

This file creates the database connection object. SQLAlchemy is an ORM (Object Relational Mapper) — it lets us work with the database using Python code instead of writing SQL queries directly.

6. Frontend Code Explanation

6.1 main.jsx — Entry Point

```
import React from 'react'
import ReactDOM from 'react-dom/client'
import App from './App.jsx'
import './index.css'
ReactDOM.createRoot(document.getElementById('root')).render(<App />)
```

This is the starting point of the React app. It takes the App component and renders (displays) it inside the HTML div with id='root' in index.html.

6.2 App.jsx — Main Component

This is the largest and most important frontend file. It contains ALL components and logic. Here are the main parts:

API Configuration

```
const API = 'https://itemflix-api.onrender.com'
```

This stores the backend URL. Every API call uses this variable. When testing locally we change this to http://127.0.0.1:5000.

State Variables (useState)

```
const [items, setItems] = useState([])
const [loading, setLoading] = useState(true)
const [search, setSearch] = useState('')
const [showForm, setShowForm] = useState(false)
const [editItem, setEditItem] = useState(null)
const [deleteItem, setDeleteItem] = useState(null)
```

```
const [viewItem, setViewItem] = useState(null)
```

State Variable	Purpose
items	Stores the list of all items fetched from API
loading	True while data is being fetched, shows spinner
search	Stores what user types in search box
showForm	True when Add/Edit form panel is open
editItem	Stores the item being edited (null = adding new)
deleteItem	Stores the item to be deleted (for confirmation modal)
viewItem	Stores the item whose detail popup is open
activePage	Tracks current page: home, mylist, or browse
activeCategory	Tracks which category filter is selected

fetchItems — GET All Items

```
const fetchItems = async () => {
  setLoading(true)
  const res = await axios.get(`${API}/items`)
  setItems(res.data.data || [])
  setLoading(false)
}
```

This function calls the Flask API to get all items. It runs automatically when the page loads (using useEffect) and also after every Add, Edit, or Delete operation to refresh the list.

useEffect — Auto Load on Start

```
useEffect(() => {
  fetchItems()
}, [])
```

useEffect with empty [] runs once when the component first loads. This automatically fetches all items when the user opens the app.

Toast Component — Notifications

Shows success (green) or error (red) messages in the top-right corner. Each toast auto-disappears after 3 seconds using setTimeout.

ItemCard Component — Each Item Card

Displays one item as a Netflix-style card with:

- Colorful gradient banner with item initials
- Item name, category badge, price in Rupees
- On hover: shows Edit and Delete buttons
- On click: opens the Detail popup modal

DetailModal Component — Item Detail Popup

Opens when user clicks any card. Shows:

- Full item details: name, description, category, price, date added, item ID
- Edit button — opens the form with existing data
- Delete button — opens confirmation modal

ItemForm Component — Add / Edit Form

A popup form that slides in from center. Used for both adding and editing items.

- If editItem is null → shows 'Add New Title' (POST request)
- If editItem has data → shows 'Edit Title' with pre-filled values (PUT request)
- Validates all inputs before sending to API

ConfirmModal Component — Delete Confirmation

A popup that asks 'Are you sure?' before deleting. Prevents accidental deletions. Only sends DELETE request when user confirms.

7. The 3 Pages Explained

Page	Nav Button	What it Shows
Home	🏠 Home	All items in a grid. Can filter by category using pill buttons. Has search bar.
My List	❤️ My List	Only shows items that have a price greater than 0. Has a red count badge.
Browse	🎬 Browse	Groups all items by their category. Shows each category as a separate row.

8. Complete Project Flow Diagram

Here is the complete flow from user action to database and back:

Step	Who	What Happens
1	User	Opens https://item-manager-rest-api.vercel.app in browser
2	Browser	Downloads and runs the React app from Vercel
3	React	Automatically sends GET request to Flask API on Render
4	Flask	Receives request, queries SQLite database for all items
5	SQLite	Returns list of items to Flask
6	Flask	Converts items to JSON and sends response to React
7	React	Receives JSON data and displays items as cards on screen
8	User	Clicks '+ Add' → fills form → clicks 'Add Title'
9	React	Sends POST request with item data to Flask API
10	Flask	Validates data, saves new item to SQLite database
11	Flask	Returns success response with new item data
12	React	Shows success toast, refreshes list, new card appears

9. Tech Stack Summary

Technology	Type	Purpose	Where Used
Python	Language	Backend programming language	Backend
Flask	Framework	Creates the REST API server and routes	Backend
SQLAlchemy	ORM	Connects Flask to SQLite database	Backend
Flask-CORS	Library	Allows React to call Flask API	Backend
SQLite	Database	Stores all item data in a file	Backend
React.js	Framework	Builds the interactive user interface	Frontend
Vite	Build Tool	Fast development server for React	Frontend
Tailwind CSS	CSS Framework	Styles all the UI components	Frontend
Axios	Library	Makes HTTP requests from React to Flask	Frontend

GitHub	Platform	Stores and version controls the code	DevOps
Render.com	Hosting	Hosts the Flask backend API (free)	DevOps
Vercel	Hosting	Hosts the React frontend (free)	DevOps

10. Live Project URLs

What	URL	Use For
Frontend App	https://item-manager-rest-api.vercel.app	Open in browser to use the app
Backend API	https://itemflix-api.onrender.com/items	Test API in Postman or browser
GitHub Repo	https://github.com/mohanraj3010/item-manager-rest-api	View all source code
Health Check	https://itemflix-api.onrender.com/health	Check if API is running

11. API Endpoints Quick Reference

Method	Endpoint	Body Required	Description
GET	/items	No	Get all items
GET	/items/1	No	Get item with ID 1
POST	/items	Yes (JSON)	Create new item
PUT	/items/1	Yes (JSON)	Update item with ID 1
DELETE	/items/1	No	Delete item with ID 1
GET	/items/search?q=phone	No	Search items by name
GET	/health	No	Check if API is running

POST / PUT Request Body Format:

```
{
  "name": "iPhone 15",
  "description": "Latest Apple phone",
  "category": "Electronics",
  "price": 75000
```

```
}
```

🎬 ITEMFLIX — Built with ❤️ using Flask + React

Developed by Mohan Raj | GitHub: mohanraj3010